thePower
FP Oficial

PROYECTO INTERMODULAR DAW

Barbería Online BarberOrz

Autor: Juan Pablo Tobar Orozco
Tutor: Raul Albiol

ÍNDICE DE CONTENIDOS

1. Introducción y Objetivos
- 1.1 Descripción del proyecto
- 1.2 Contexto y motivación
- 1.3 Objetivos generales
- 1.4 Objetivos específicos
- 1.5 Alcance del proyecto
- 1.6 Planificación temporal

2. Análisis y Requisitos
- 2.1 Estudio previo del mercado
- 2.2 Requisitos funcionales
- 2.3 Requisitos no funcionales
- 2.4 Casos de uso
- 2.5 Diagrama de flujo general

3. Diseño de la Base de Datos
- 3.1 Modelo Entidad-Relación
- 3.2 Tabla: usuarios
- 3.3 Tabla: servicios
- 3.4 Tabla: citas
- 3.5 Tabla: productos
- 3.6 Tabla: pedidos
- 3.7 Tabla: pedidos_linea
- 3.8 Relaciones e integridad referencial
- 3.9 Script SQL de creación
- 3.10 Datos iniciales de prueba

4. Desarrollo Frontend
- 4.1 Tecnologías utilizadas
- 4.2 Arquitectura y estructura de archivos
- 4.3 Sistema de diseño y variables CSS
- 4.4 Página principal (index.html)
- 4.5 Sistema de autenticación
- 4.6 Reserva de citas (reservas.html)
- 4.7 Tienda online y carrito
- 4.8 Panel de usuario (perfil.html)
- 4.9 Panel de administración (admin.html)

5. Desarrollo Backend
- 5.1 Tecnologías y dependencias
- 5.2 Arquitectura del servidor
- 5.3 Configuración y variables de entorno
- 5.4 Módulo de autenticación
- 5.5 Módulo de reservas y citas
- 5.6 Módulo de tienda
- 5.7 Seguridad: JWT y bcrypt

5.8 Middleware de protección de rutas
6. Manual de Usuario
6.1 Requisitos del sistema
6.2 Instalación paso a paso
6.3 Guía del cliente
6.4 Guía del administrador
7. Conclusiones
7.1 Resultados obtenidos
7.2 Dificultades y soluciones
7.3 Competencias adquiridas
7.4 Mejoras futuras
7.5 Valoración personal
Bibliografía y Referencias

1. Introducción y Objetivos

1.1 Descripción del proyecto

Barber O.R.Z Studio es una aplicación web *full stack* orientada a la gestión integral de una barbería profesional. La herramienta permite digitalizar los principales procesos del negocio, incluyendo la reserva online de citas con control de disponibilidad en tiempo real, la venta de productos de cuidado personal mediante una tienda integrada, la gestión de perfiles e historiales de clientes, así como la administración global del establecimiento a través de un panel de control específico para el administrador.

Desde el punto de vista técnico,. El frontend se desarrolla como una aplicación multipágina (MPA) utilizando HTML5, CSS3 y JavaScript ES6+, sin el uso de frameworks adicionales. Por su parte, el backend consiste en una API REST implementada con Node.js y Express, conectada a una base de datos MySQL. La comunicación entre ambas capas se realiza de forma asíncrona mediante la Fetch API del navegador, empleando el formato JSON para el intercambio de datos.

El proyecto abarca el ciclo completo de desarrollo de software, desde el análisis de requisitos y el diseño de la base de datos, hasta la implementación del backend, el desarrollo del frontend y la correspondiente documentación. Como resultado, se obtiene una aplicación plenamente funcional, susceptible de ser implantada en un entorno real con mínimas adaptaciones.

1.2 Contexto y motivación

El sector de la peluquería y barbería en España está compuesto principalmente por pequeños negocios independientes. Según datos del INE, existen más de 45.000 establecimientos de peluquería y estética en España, la mayoría de ellos microempresas o autónomos. La gran mayoría de estos negocios gestionan sus citas de manera tradicional: por teléfono, de forma presencial o mediante aplicaciones genéricas de mensajería como WhatsApp o Instagram. Esta situación genera múltiples ineficiencias:

- Pérdida de clientes potenciales fuera del horario de atención telefónica, especialmente por las noches y los fines de semana.
- Imposibilidad de que el cliente consulte la disponibilidad horaria sin hacer una llamada.
- Ausencia de un historial organizado de citas y clientes accesible en todo momento.
- Falta de un canal de venta digital para productos de la barbería.
- Dependencia de plataformas genéricas de terceros (Treatwell, Booksy) que cobran comisiones elevadas y no permiten personalización de la identidad visual del negocio.

Una aplicación web propia resuelve todos estos problemas, permitiendo al negocio estar disponible las 24 horas del día para que los clientes puedan reservar, cancelar y comprar sin depender del horario de apertura. Además, al ser un desarrollo propio, no existen costes

de comisión por reserva y la identidad visual del negocio queda completamente integrada en la experiencia del usuario.

Desde el punto de vista académico, este proyecto representa la oportunidad ideal de aplicar de forma integrada y coherente todas las competencias desarrolladas a lo largo del ciclo formativo de Desarrollo de Aplicaciones Web: bases de datos, programación del servidor, desarrollo de interfaces web y metodología de desarrollo de software.

1.3 Objetivos generales

El objetivo principal de este proyecto es diseñar y desarrollar una aplicación web *full stack* funcional que permita digitalizar la gestión integral de una barbería, evidenciando el dominio de las tecnologías fundamentales del desarrollo web tanto en el lado del servidor como en el del cliente.

Para alcanzar este propósito, se plantea la creación de un sistema orientado a dos perfiles de usuario claramente diferenciados. Por un lado, el cliente, que requiere una interfaz sencilla e intuitiva para la reserva de citas y la compra de productos. Por otro, el administrador del negocio, que necesita disponer de herramientas eficaces para la gestión de citas, pedidos y catálogo, facilitando así el control y la organización del establecimiento.

1.4 Objetivos específicos

A partir del objetivo general se derivan los siguientes objetivos específicos, cada uno de los cuales se ha cumplido a lo largo del desarrollo:

- OE-01: Diseñar e implementar un modelo de base de datos relacional en MySQL que almacene de forma eficiente toda la información del sistema, garantizando la integridad referencial.
- OE-02: Desarrollar una API REST con Node.js y Express que exponga los recursos del sistema de forma clara y segura, siguiendo los principios REST.
- OE-03: Implementar un sistema de autenticación seguro basado en JSON Web Tokens (JWT) con cifrado de contraseñas mediante bcrypt.
- OE-04: Crear un sistema de reservas online con control de disponibilidad horaria en tiempo real, evitando solapamientos de citas.
- OE-05: Desarrollar una tienda online con catálogo de productos, carrito de compras persistente en localStorage y proceso de pedido completo.
- OE-06: Implementar un panel de usuario donde el cliente pueda consultar y gestionar su historial de citas y pedidos.
- OE-07: Construir un panel de administración con gestión completa de citas, pedidos y productos (operaciones CRUD).
- OE-08: Diseñar una interfaz visual coherente y profesional con identidad corporativa propia, completamente responsiva.
- OE-09: Garantizar la seguridad de la aplicación mediante validación de datos, protección de rutas y control de roles.
- OE-10: Documentar el proyecto de forma completa en esta memoria de TFG.

1.5 Alcance del proyecto

El alcance define con precisión qué funcionalidades están incluidas en el proyecto entregado y cuáles quedan fuera del mismo, como posibles mejoras futuras.

Funcionalidades incluidas en el alcance:

- Registro de usuarios con nombre, apellidos, teléfono, email y contraseña.
- Inicio de sesión con email y contraseña y generación de token JWT.
- Diferenciación de roles: cliente y administrador.
- Catálogo de servicios de barbería con nombre, descripción, precio y duración.
- Reserva de citas en tres pasos: selección de servicio, fecha y hora, y confirmación.
- Control de disponibilidad horaria en tiempo real mediante consulta a la API.
- Cancelación de citas propias por parte del cliente.
- Catálogo de productos con imagen, nombre, descripción y precio.
- Carrito de compras persistente en localStorage con control de cantidades.
- Proceso de compra completo con registro del pedido en la base de datos.
- Panel de usuario con historial de citas y pedidos.
- Panel de administración con gestión de citas (estados), pedidos y productos (CRUD).
- Navbar dinámica que muestra el nombre del usuario autenticado.
- Diseño responsivo para móvil, tablet y escritorio.

Funcionalidades fuera del alcance (mejoras futuras):

- Pasarela de pago real (Stripe, PayPal u otras).
- Envío de notificaciones por email o SMS.
- Sistema de valoraciones y reseñas de servicios.
- Integración con Google Calendar.
- Gestión de múltiples barberos con agendas independientes.
- Aplicación móvil nativa.

1.6 Planificación temporal

El proyecto se desarrolló durante el segundo trimestre del curso académico 2025/2026, con una duración total aproximada de ocho semanas de trabajo. A continuación se detalla la distribución temporal por fases:

Fase	Semana	Duración	Tareas principales
Fase 1	1	1 semana	Análisis de requisitos, estudio de tecnologías, diseño de wireframes y esquema de base de datos.

Fase	Semana	Duración	Tareas principales
Fase 2	2-3	2 semanas	Creación del esquema SQL, configuración del servidor Express, implementación de la API REST.
Fase 3	3	1 semana	Sistema de autenticación: registro, login, JWT, middleware de roles.
Fase 4	4	1 semana	Frontend: página principal (index.html), diseño visual, CSS y JavaScript.
Fase 5	5-6	2 semanas	Frontend: página de reservas, carrito de compras, proceso de pedido.
Fase 6	6	1 semana	Paneles de usuario y administrador. CRUD de productos.
Fase 7	7	1 semana	Pruebas de integración, corrección de errores, ajustes de diseño responsive.
Fase 8	8	1 semana	Redacción de la memoria del TFG y documentación final.

2. Análisis y Requisitos

2.1 Estudio previo del mercado

Antes de comenzar el desarrollo se realizó un análisis de las soluciones existentes en el mercado para la gestión de barberías y salones de belleza. Se estudiaron en detalle tres plataformas líderes: Treatwell, Booksy y Fresha.

Plataforma	Ventajas	Inconvenientes	Coste
Treatwell	Gran visibilidad, muchos usuarios registrados	Comisión del 20-25% por reserva, marca propia diluida	Comisión por reserva
Booksy	Gestión de agenda completa	Cuota mensual, poco personalizable	~30€/mes
Fresha	Gratuito para el negocio	Cobra al cliente, interfaz compleja	0€ + comisiones
Barber O.R.Z	100% personalizado, sin comisiones, integración tienda	Requiere desarrollo y mantenimiento	Solo desarrollo

Del análisis se extraen las siguientes conclusiones que guían el diseño de la aplicación:

- La identidad visual del negocio debe estar completamente integrada: los colores, tipografías y la experiencia de usuario deben reflejar la marca de la barbería.
- La funcionalidad de reservas debe ser tan sencilla como sea posible: máximo tres pasos.
- Integrar la tienda de productos y las reservas en el mismo sistema aumenta el valor percibido y el ticket medio del cliente.
- El panel de administración debe ser práctico y eficiente para que el propietario pueda gestionar el negocio sin formación técnica.

2.2 Requisitos funcionales

Los requisitos funcionales describen con precisión las funcionalidades que debe implementar el sistema. Se organizan en módulos y se identifican con un código para facilitar su trazabilidad:

RF-01 – Módulo de autenticación

ID	Nombre	Descripción
RF-01.1	Registro	El sistema permite crear una cuenta con nombre, apellidos, teléfono (opcional), email y contraseña. El email debe ser único en el sistema.
RF-01.2	Login	El sistema verifica las credenciales y, si son correctas, genera un token JWT firmado con caducidad de 7 dias.
RF-01.3	Logout	El sistema elimina el token JWT y los datos de sesión del localStorage del navegador.
RF-01.4	Roles	El sistema diferencia dos roles: "cliente" (acceso a reservas, tienda y perfil) y "admin" (acceso al panel de administración).
RF-01.5	Rutas protegidas	Las rutas privadas verifican el token antes de permitir el acceso. Sin token válido devuelven error 401.
RF-01.6	Control de rol	Las rutas de administración verifican además que el rol sea "admin". Sin el rol correcto devuelven error 403.

RF-02 – Módulo de servicios

ID	Nombre	Descripción
RF-02.1	Listar servicios	El sistema devuelve el catálogo completo de servicios con id, nombre, descripción, precio y duración. Accesible sin autenticación.
RF-02.2	Visualización	Los servicios se muestran en la página principal y en la página de reservas con imagen representativa.

RF-03 – Módulo de reservas

ID	Nombre	Descripción
RF-03.1	Selección de servicio	El cliente elige el servicio deseado de entre los disponibles en el catálogo.
RF-03.2	Selección de fecha	El sistema muestra un selector de fecha con restricción de fecha mínima = hoy. No se permiten fechas pasadas.
RF-03.3	Disponibilidad horaria	Al seleccionar la fecha, el sistema obtiene de la API los horarios ocupados y muestra cuáles están disponibles.
RF-03.4	Bloqueo de horas pasadas	Si la fecha seleccionada es el día de hoy, las horas ya transcurridas se muestran deshabilitadas.
RF-03.5	Confirmación	Antes de crear la cita el sistema muestra un resumen completo: servicio, fecha formateada, hora, duración y precio.
RF-03.6	Crear reserva	El sistema registra la cita con estado "pendiente" y el precio cobrado en el momento de la reserva.
RF-03.7	Anti-solapamiento	El servidor verifica que no exista ninguna cita solapada antes de guardar la nueva reserva.
RF-03.8	Cancelar cita	El cliente puede cancelar sus propias citas en estado "pendiente" o "confirmada".

ID	Nombre	Descripción
RF-03.9	Historial	El cliente puede consultar todas sus citas con el estado, servicio, fecha, hora y precio.

RF-04 – Módulo de tienda

ID	Nombre	Descripción
RF-04.1	Catálogo	El sistema muestra los productos con imagen, nombre, descripción y precio. Accesible sin autenticación.
RF-04.2	Añadir al carrito	El cliente puede añadir productos al carrito. Si el producto ya está, se incrementa la cantidad.
RF-04.3	Ver carrito	El carrito muestra imagen, nombre, precio unitario, cantidad y subtotal de cada producto.
RF-04.4	Modificar cantidad	El cliente puede aumentar o disminuir la cantidad. Si llega a 0, el producto se elimina del carrito.
RF-04.5	Eliminar producto	El cliente puede eliminar un producto del carrito mediante un botón de borrado.
RF-04.6	Total en tiempo real	El total del carrito se recalcula automáticamente al modificar cantidades.
RF-04.7	Finalizar compra	El sistema envía el pedido al backend, registra las líneas y limpia el carrito. Requiere autenticación.
RF-04.8	Historial de pedidos	El cliente puede ver sus pedidos anteriores con fecha, total y detalle de productos.

RF-05 – Panel de administración

ID	Nombre	Descripción
RF-05.1	Estadísticas	El panel muestra tarjetas con el total de citas registradas, pedidos realizados y productos activos.
RF-05.2	Ver todas las citas	El admin puede ver todas las citas del sistema con cliente, servicio, fecha, hora, precio y estado.
RF-05.3	Filtrar citas	El admin puede filtrar las citas por estado: pendiente, confirmada, completada o cancelada.
RF-05.4	Cambiar estado de cita	El admin puede cambiar el estado de cualquier cita mediante un selector desplegable en la tabla.
RF-05.5	Ver todos los pedidos	El admin puede ver todos los pedidos con el nombre del cliente, fecha y total.
RF-05.6	Crear producto	El admin puede crear nuevos productos mediante un formulario modal con validación.
RF-05.7	Editar producto	El admin puede modificar el nombre, descripción, precio y stock de cualquier producto.
RF-05.8	Eliminar producto	El admin puede eliminar productos del catálogo tras confirmación.

2.3 Requisitos no funcionales

ID	Categoría	Descripción
RNF-01	Usabilidad	La interfaz debe ser intuitiva y no requerir formación previa. El proceso de reserva debe completarse en menos de 2 minutos.
RNF-02	Responsividad	La aplicación debe funcionar correctamente y de forma usable en dispositivos móviles (min. 320px), tablets y escritorio.
RNF-03	Rendimiento	Los endpoints de la API deben responder en menos de 500ms en condiciones normales con la base de datos local.
RNF-04	Seguridad	Las contraseñas se almacenan cifradas con bcrypt. Los tokens JWT caducan a los 7 dias. Las rutas privadas están protegidas.
RNF-05	Disponibilidad	El sistema funciona completamente en local sin dependencia de servicios externos en tiempo de ejecución.
RNF-06	Mantenibilidad	El código está organizado en módulos independientes, con separación de capas y comentado adecuadamente.
RNF-07	Compatibilidad	Compatible con Chrome 90+, Firefox 88+ y Edge 90+. No se requiere soporte para IE11.
RNF-08	Escalabilidad	La arquitectura debe permitir añadir nuevos módulos o endpoints sin refactorización mayor del código existente.
RNF-09	Portabilidad	La aplicación puede ejecutarse en cualquier sistema operativo con Node.js y MySQL instalados.

2.4 Casos de uso

Los casos de uso describen las interacciones entre los actores del sistema y la aplicación. El sistema tiene dos actores principales: el Cliente (usuario registrado con rol "cliente") y el Administrador (usuario con rol "admin"). Existe además un actor secundario: el Usuario no registrado, que puede consultar la información pública pero no puede reservar ni comprar.

CU-01: Registrarse en el sistema

Actor principal	Usuario no registrado
Precondición	El usuario no tiene cuenta en el sistema
Flujo normal	1. El usuario hace clic en "Registro" en la navbar. 2. Rellena el formulario con nombre, apellidos, teléfono, email y contraseña.3. Confirma la contraseña en el segundo campo.4. Pulsa "Crear cuenta". 5. El sistema valida los datos en el cliente.6. El sistema envía la petición al servidor.7. El servidor verifica que el email no existe.8. El servidor cifra la contraseña y registra el usuario.9. El sistema redirige al login con mensaje de éxito.
Flujo alternativo 1	Si el email ya existe: el servidor devuelve error 409 y el sistema muestra "Este email ya está registrado".

Flujo alternativo 2	Si las contraseñas no coinciden: el cliente muestra error sin enviar la petición.
Postcondición	El usuario tiene una cuenta creada con rol "cliente" en la base de datos.

CU-02: Iniciar sesión

Actor principal	Usuario registrado
Precondición	El usuario tiene una cuenta activa en el sistema
Flujo normal	1. El usuario accede a la página de login.2. Introduce su email y contraseña.3. Pulsa "Entrar al Studio".4. El servidor verifica las credenciales con bcrypt.5. El servidor genera un token JWT (8 horas).6. El cliente guarda el token y los datos en localStorage. 7. El sistema redirige según el rol: admin→admin.html, cliente→index.html.
Flujo alternativo	Si las credenciales son incorrectas: el servidor devuelve 401 y se muestra un mensaje de error.
Postcondición	El usuario está autenticado. La navbar muestra su nombre.

CU-03: Reservar una cita

Actor principal	Cliente autenticado
Precondición	El usuario ha iniciado sesión con rol "cliente"
Flujo normal	1. El cliente accede a la página de reservas.2. Selecciona el servicio deseado (Paso 1).3. Selecciona la fecha en el calendario (Paso 2).4. El sistema carga los horarios disponibles para esa fecha.5. El cliente selecciona una hora disponible.6. El sistema muestra el resumen de la cita (Paso 3).7. El cliente pulsa "Confirmar cita".8. El sistema crea la cita y muestra mensaje de éxito.
Flujo alternativo 1	Si el usuario no está autenticado: se redirige al login.
Flujo alternativo 2	Si la hora se ocupa entre la selección y la confirmación: el servidor devuelve error y pide nueva selección.
Postcondición	La cita queda registrada en estado "pendiente".

CU-04: Cancelar una cita

Actor principal	Cliente autenticado
Precondición	El cliente tiene al menos una cita en estado "pendiente" o "confirmada"
Flujo normal	1. El cliente accede a su panel de perfil.2. Hace clic en la pestaña "Mis Citas".3. Localiza la cita que desea cancelar.4. Pulsa el botón "Cancelar".5. El sistema envía la petición PUT al servidor.6. El servidor cambia el estado a "cancelada".7. La tarjeta de la cita se actualiza visualmente.
Flujo alternativo	Si la cita ya está cancelada o completada: el botón no aparece.

Postcondición	El estado de la cita pasa a "cancelada" y la hora queda libre para nuevas reservas.

CU-05: Realizar una compra

Actor principal	Cliente autenticado
Precondición	El usuario ha iniciado sesión y tiene productos en el carrito
Flujo normal	1. El cliente añade productos con el botón "+ Carrito".2. Accede a la página del carrito.3. Revisa los productos, cantidades y el total.4. Pulsa "Finalizar compra".5. El sistema envía el pedido al backend con el token JWT.6. El servidor registra el pedido y las líneas en la BD.7. El carrito se vacía y se muestra confirmación.
Flujo alternativo	Si el usuario no está autenticado al finalizar: se redirige al login.
Postcondición	El pedido queda registrado. El carrito queda vacío.

CU-06: Gestionar productos (Admin)

Actor principal	Administrador
Precondición	El admin ha iniciado sesión con rol "admin"
Flujo – Crear	1. Pulsa "+ Nuevo producto".2. Rellena nombre, descripción, precio y stock.3. Pulsa "Guardar".4. El producto aparece en la tabla y en la tienda.
Flujo – Editar	1. Pulsa "Editar" en la fila del producto.2. El modal se abre con los datos actuales.3. Modifica los campos necesarios.4. Pulsa "Guardar". Los datos se actualizan.
Flujo – Eliminar	1. Pulsa "Borrar" en la fila del producto.2. Confirma en el diálogo.3. El producto desaparece de la tabla y del catálogo.
Postcondición	El catálogo de productos refleja los cambios realizados.

2.5 Diagrama de flujo general

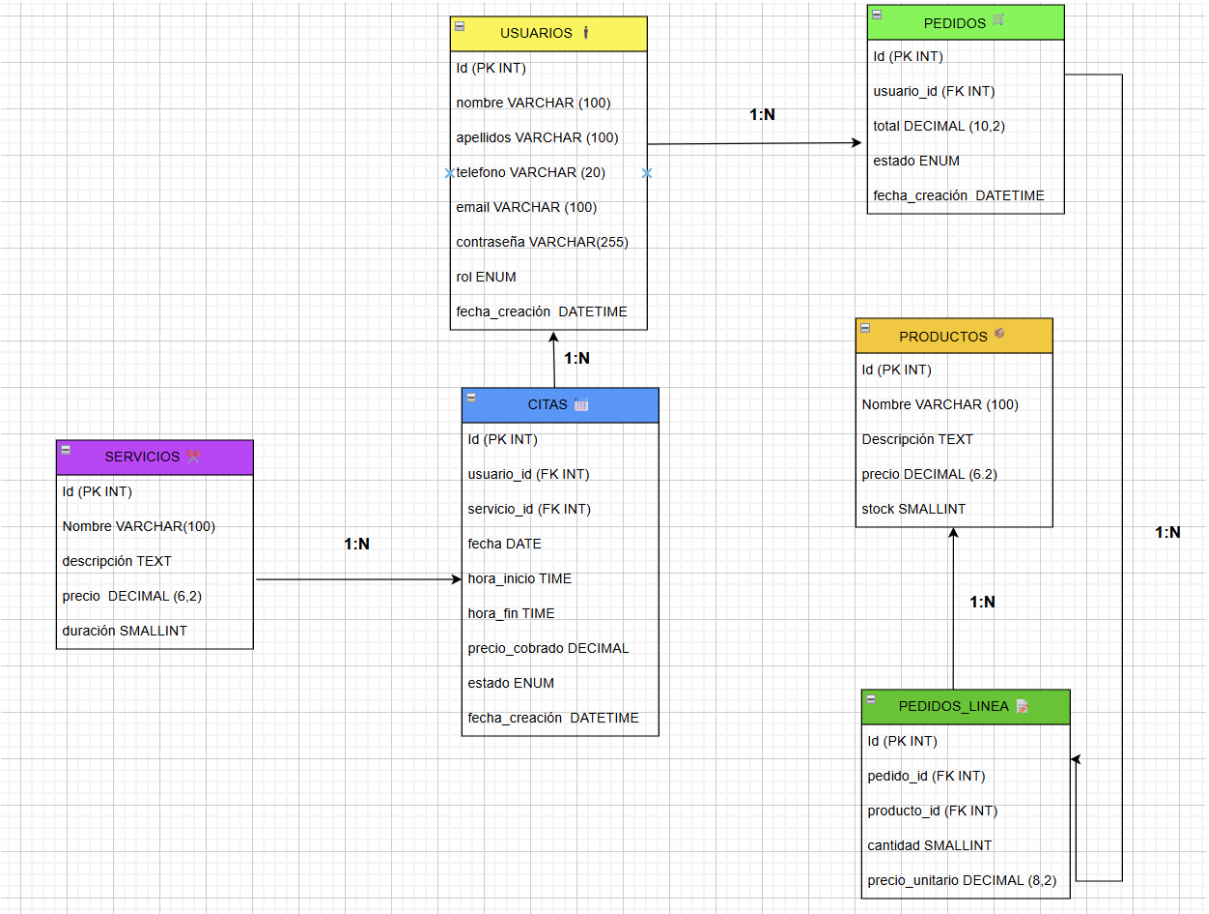
El flujo de navegación de la aplicación sigue el esquema descrito a continuación. Todos los usuarios (autenticados o no) pueden acceder a la página principal, consultar servicios y ver productos. Para las acciones que requieren identidad, el sistema redirige al login:

- Página principal (index.html) → punto de entrada. Acceso público a servicios y catálogo de productos.
- Login (login.html) → Autenticación. Redirige según rol: admin.html o index.html.
- Registro (registro.html) → Creación de cuenta. Redirige a login tras el registro exitoso.
- Reservas (reservas.html) → Flujo de 3 pasos. Requiere token válido. Sin token → redirige a login.
- Carrito (carrito.html) → Gestión del carrito. Consulta pública, finalizar compra requiere token.

- Perfil (perfil.html) → Panel del cliente. Requiere token válido. Sin token → redirige a login.
- Admin (admin.html) → Panel de administración. Requiere token con rol "admin". Sin rol → redirige a index con alerta.

3. Diseño de la Base de Datos

3.1 Modelo Entidad-Relación



El modelo de datos de la aplicación está compuesto por seis entidades que representan los conceptos fundamentales del negocio. El diseño sigue las reglas de normalización hasta la Tercera Forma Normal (3FN) para eliminar redundancias y garantizar la integridad de los datos.

Las entidades del modelo son:

- USUARIOS: central del modelo. Representa tanto a los clientes como al administrador del sistema.
- SERVICIOS: catálogo de servicios de barbería. Entidad de referencia que no cambia frecuentemente.
- CITAS: entidad principal del negocio. Representa una reserva concreta de un usuario para un servicio en una fecha y hora.
- PRODUCTOS: catálogo de artículos de la tienda online.
- PEDIDOS: cabecera de cada pedido. Agrupa las líneas del pedido y registra el total.
- PEDIDOS_LINEA: detalle de cada pedido. Una fila por cada producto comprado en un pedido.

Las cardinalidades de las relaciones son:

- USUARIOS ↔ CITAS: un usuario puede tener muchas citas (1:N). Una cita pertenece a un único usuario.
- SERVICIOS ↔ CITAS: un servicio puede aparecer en muchas citas (1:N). Una cita tiene un único servicio.
- USUARIOS ↔ PEDIDOS: un usuario puede tener muchos pedidos (1:N). Un pedido pertenece a un único usuario.
- PEDIDOS ↔ PEDIDOS_LINEA: un pedido tiene muchas líneas (1:N). Una línea pertenece a un único pedido.
- PRODUCTOS ↔ PEDIDOS_LINEA: un producto puede aparecer en muchas líneas (1:N). Una línea referencia un único producto.

3.2 Tabla: usuarios

Almacena los datos de registro de todos los usuarios del sistema. La columna "rol" usa el tipo ENUM de MySQL para garantizar que solo se puedan asignar los valores permitidos.

Campo	Tipo	Restricción	Default	Descripción
id	INT UNSIGNED	PK, AUTO_INCREMENT	-	Identificador único autoincremental
nombre	VARCHAR(100)	NOT NULL	-	Nombre de pila del usuario
apellidos	VARCHAR(150)	NULL	NULL	Apellidos del usuario
telefono	VARCHAR(20)	NULL	NULL	Número de teléfono de contacto
email	VARCHAR(150)	UNIQUE, NOT NULL	-	Correo electrónico. Debe ser único en el sistema.
contraseña	VARCHAR(255)	NOT NULL	-	Hash bcrypt de 60 caracteres. Nunca el texto plano.
rol	ENUM(cliente,admin)	NOT NULL	cliente	Rol del usuario en el sistema
fecha_creacion	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	Fecha y hora de registro automática

3.3 Tabla: servicios

Catálogo de los servicios que ofrece la barbería. Esta tabla es estática durante el uso normal de la aplicación. El campo duración se almacena en minutos para facilitar los cálculos de hora_fin en las reservas.

Campo	Tipo	Restricción	Default	Descripción
id	INT UNSIGNED	PK, AUTO_INCREMENT	-	Identificador único
nombre	VARCHAR(100)	NOT NULL	-	Nombre del servicio (p. ej. "Corte + Barba")
descripción	TEXT	NULL	NULL	Descripción larga del servicio
precio	DECIMAL(8,2)	NOT NULL	-	Precio en euros con dos decimales
duración	INT UNSIGNED	NOT NULL	-	Duración en minutos (p. ej. 50 para 50 min)

3.4 Tabla: citas

Entidad central del módulo de reservas. Registra cada reserva con todos sus datos. El campo precio_cobrado guarda el precio del servicio en el momento de la reserva, protegiéndose de futuros cambios de precio. La hora_fin se calcula en el servidor sumando la duración del servicio a la hora_inicio.

Campo	Tipo	Restricción	Default	Descripción
id	INT UNSIGNED	PK, AUTO_INCREMENT	-	Identificador único
usuario_id	INT UNSIGNED	FK → usuarios.id	-	Cliente que realiza la reserva
servicio_id	INT UNSIGNED	FK → servicios.id	-	Servicio que se reserva
fecha	DATE	NOT NULL	-	Fecha de la cita (YYYY-MM-DD)
hora_inicio	TIME	NOT NULL	-	Hora de inicio de la cita (HH:MM:SS)
hora_fin	TIME	NOT NULL	-	Hora de fin calculada por el servidor
precio_cobrado	DECIMAL(8,2)	NOT NULL	-	Precio del servicio en el momento de la reserva
estado	ENUM(...)	NOT NULL	pendiente	pendiente / confirmada / completada / cancelada
fecha_creación	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	Fecha de creación del registro

3.5 Tabla: productos

Catálogo de productos de la tienda online. El campo stock permite al administrador controlar las unidades disponibles de cada producto, aunque en esta versión del proyecto no se descuenta el stock automáticamente al realizar un pedido (mejora futura).

Campo	Tipo	Restricción	Default	Descripción
id	INT UNSIGNED	PK, AUTO_INCREMENT	-	Identificador único
nombre	VARCHAR(100)	NOT NULL	-	Nombre del producto
descripcion	TEXT	NULL	NULL	Descripción del producto
precio	DECIMAL(8,2)	NOT NULL	-	Precio de venta en euros
stock	INT UNSIGNED	NOT NULL	0	Unidades disponibles en inventario

3.6 Tabla: pedidos

Cabecera de cada transacción de compra. Agrupa las líneas del pedido y registra el total calculado como suma de (precio_unitario × cantidad) de cada línea. El campo estado permite al administrador gestionar el ciclo de vida del pedido.

Campo	Tipo	Restricción	Default	Descripción
id	INT UNSIGNED	PK, AUTO_INCREMENT	-	Identificador único
usuario_id	INT UNSIGNED	FK → usuarios.id	-	Cliente que realiza el pedido
total	DECIMAL(10,2)	NOT NULL	-	Suma total del pedido en euros
estado	ENUM(...)	NOT NULL	pendiente	pendiente / enviado / entregado / cancelado
fecha_creacion	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	Fecha y hora del pedido

3.7 Tabla: pedidos_linea

Tabla de detalle de los pedidos. Cada fila representa un producto comprado en un pedido concreto. Al igual que con las citas, se guarda el precio_unitario en el momento de la compra para protegerse de cambios de precio futuros.

Campo	Tipo	Restricción	Default	Descripción
id	INT UNSIGNED	PK, AUTO_INCREMENT	-	Identificador único
pedido_id	INT UNSIGNED	FK → pedidos.id	-	Pedido al que pertenece la línea
producto_id	INT UNSIGNED	FK → productos.id	-	Producto comprado
cantidad	INT UNSIGNED	NOT NULL	-	Unidades compradas de este producto
precio_unitario	DECIMAL(8,2)	NOT NULL	-	Precio del producto en el momento de la compra

3.8 Relaciones e integridad referencial

Las claves foráneas garantizan que no puedan existir registros huérfanos en la base de datos. Se han definido con las siguientes acciones en cascada:

Clave foránea	Referencia	ON DELETE	Justificación
citas.usuario_id	usuarios.id	CASCADE	Al borrar un usuario se borran sus citas
citas.servicio_id	servicios.id	RESTRICT	No se puede borrar un servicio con citas
pedidos.usuario_id	usuarios.id	CASCADE	Al borrar un usuario se borran sus pedidos
pedidos_linea.pedido_id	pedidos.id	CASCADE	Al borrar un pedido se borran sus líneas
pedidos_linea.producto_id	productos.id	RESTRICT	No se puede borrar un producto con pedidos

3.9 Script SQL de creación

El esquema de la base de datos se crea mediante un script SQL que incluye la creación de la base de datos, todas las tablas con sus restricciones y claves foráneas. A continuación se muestra un fragmento representativo del script:

```
CREATE DATABASE IF NOT EXISTS barberia_db CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
```

```
USE barberia_db;

CREATE TABLE usuarios (
  id          INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre      VARCHAR(100) NOT NULL,
  apellidos   VARCHAR(150),
  telefono    VARCHAR(20),
  email       VARCHAR(150) NOT NULL UNIQUE,
  contraseña  VARCHAR(255) NOT NULL,
  rol         ENUM("cliente","admin") NOT NULL DEFAULT "cliente",
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE citas (
  id             INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  usuario_id     INT UNSIGNED NOT NULL,
  servicio_id    INT UNSIGNED NOT NULL,
  fecha          DATE NOT NULL,
  hora_inicio    TIME NOT NULL,
  hora_fin       TIME NOT NULL,
  precio_cobrado DECIMAL(8,2) NOT NULL,
  estado         ENUM("pendiente","confirmada","completada","cancelada")
DEFAULT "pendiente",
  created_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (usuario_id) REFERENCES usuarios(id) ON DELETE CASCADE,
  FOREIGN KEY (servicio_id) REFERENCES servicios(id) ON DELETE RESTRICT
);
```

3.10 Datos iniciales de prueba

El script incluye datos de prueba (INSERT) para poder verificar el funcionamiento completo de la aplicación sin necesidad de registrar datos manualmente:

Usuarios de prueba:

Email	Contraseña	Rol	Nombre completo
admin@barberia.com	admin123	admin	Administrador del sistema
juan@test.com	123456	cliente	Juan García López
pedro@gmail.com	123456	cliente	Pedro Sánchez Martín

Servicios disponibles:

Servicio	Precio	Duración	Descripción
Corte de pelo	12,00 €	30 min	Corte clásico a tijera o máquina con acabado profesional
Arreglo de barba	8,00 €	20 min	Perfilado y arreglo con navaja de afeitar
Corte + Barba	18,00 €	50 min	Servicio completo: corte de pelo y arreglo de barba

Servicio	Precio	Duración	Descripción
Afeitado clásico	15,00 €	40 min	Afeitado tradicional con navaja, toalla caliente y aftershave

4. Desarrollo Frontend

4.1 Tecnologías utilizadas

El frontend está desarrollado íntegramente con tecnologías web estándar sin ningún framework adicional. Esta decisión fue deliberada: el objetivo académico de consolidar los fundamentos del desarrollo web y demostrar el dominio de HTML, CSS y JavaScript puro justifica completamente esta elección frente al uso de React, Vue o Angular.

Tecnología	Versión	Uso en el proyecto
HTML5	Estándar W3C	Estructura semántica de las 7 páginas de la aplicación
CSS3	Estándar W3C	Diseño visual completo: variables, flexbox, grid, animaciones, media queries
JavaScript	ES6+ (2015+)	Lógica de negocio, manipulación del DOM, llamadas a la API, validaciones
Fetch API	Nativa navegador	Peticiones HTTP asíncronas al backend con async/await
localStorage	Nativa navegador	Persistencia del token JWT, datos del usuario y carrito de compras
Google Fonts	API externa CDN	Tipografías Cinzel Decorative, Cinzel y Raleway
Live Server	VS Code Extension	Servidor de desarrollo local con recarga automática
Variables CSS	CSS Custom Properties	Sistema de tokens de diseño para colores y tamaños

4.2 Arquitectura y estructura de archivos

El proyecto sigue una arquitectura MPA (Multi-Page Application) donde cada funcionalidad principal tiene su propia página HTML. Esta arquitectura es la más adecuada para el contexto del proyecto por su simplicidad, facilidad de mantenimiento y compatibilidad con el enfoque sin frameworks.

La separación de responsabilidades se aplica estrictamente: cada página tiene su propio archivo HTML para la estructura, su hoja de estilos CSS para la presentación y su script JavaScript para el comportamiento. El archivo main.js contiene funciones compartidas entre páginas como la gestión del carrito y la actualización de la navbar.

```
frontend/
├── pages/
│   ├── index.html       → Página principal (landing page)
│   ├── login.html       → Formulario de inicio de sesión
│   ├── registro.html    → Formulario de registro de usuarios
│   ├── reservas.html    → Sistema de reserva de citas (3 pasos)
│   ├── carrito.html     → Carrito de compras y proceso de pedido
│   ├── perfil.html      → Panel personal del cliente
```

```
│   └── admin.html       → Panel de administración
├── css/
│   ├── style.css        → Estilos página principal
│   ├── login.css        → Estilos login y registro
│   ├── reservas.css     → Estilos sistema de reservas
│   ├── carrito.css      → Estilos carrito
│   ├── perfil.css       → Estilos panel usuario
│   └── admin.css        → Estilos panel administrador
    └── registro.css     → Estilos panel de registro
├── js/
│   ├── main.js          → JS principal + carrito + navbar dinámica
│   ├── login.js         → Lógica de autenticación login/registro
│   ├── reservas.js      → Lógica del sistema de reservas
│   ├── carrito.js       → Lógica del carrito de compras
│   ├── perfil.js        → Lógica del panel de usuario
│   └── admin.js         → Lógica del panel de administrador
    └── registro.js      → Lógica del panel de registro

└── img/
    ├── barberia dentro dorada.jpeg
    ├── Barberia Logo.jpeg
    ├── tapper fade.jpg
    ├── barba1.jpg
    ├── corte y barba.jpg
    ├── Buzz cut.jpg
    ├── cera (60ml).jpg
    ├── after shave 400ml.jpg
    ├── desinfectante 5 en 1 400ml.png
    └── polvos volumen 20g.jpg
```

4.3 Sistema de diseño y variables CSS

La coherencia visual entre todas las páginas de la aplicación se logra mediante un sistema de variables CSS (Custom Properties) definidas en el selector :root de cada hoja de estilos. Esto permite cambiar toda la paleta de colores de la aplicación modificando un único bloque de código.

Paleta de colores

Variable CSS	Valor hex	Uso principal
--gold	#c9a84c	Color principal. Bordes activos, iconos, textos destacados, botones.
--gold-light	#e8c96a	Variante clara. Gradientes, efectos hover, brillos.
--gold-dark	#8b6914	Variante oscura. Fondos de botones, gradientes profundos.
--black	#0a0a0a	Fondo principal de toda la aplicación.
--black-card	#0d0d08	Fondo de tarjetas y contenedores elevados.
--black-lighter	#141414	Fondo de secciones alternadas.
--white	#f5f0e8	Blanco cálido. Color del texto principal sobre fondo oscuro.

Variable CSS	Valor hex	Uso principal
--gray	#888888	Textos secundarios, placeholders, metadatos.

Tipografías

Se importan tres familias tipográficas de Google Fonts que configuran tres niveles jerárquicos bien diferenciados:

- Cinzel Decorative (pesos 700 y 900): para el nombre de la barbería y los títulos principales de sección. Es una tipografía romana de gran impacto visual que aporta elegancia y personalidad a la marca.
- Cinzel (pesos 400, 600 y 700): para subtítulos, etiquetas, botones y elementos de navegación. Mantiene el carácter clásico con mayor legibilidad en tamaños menores.
- Raleway (pesos 300, 400 y 500): para el cuerpo del texto, descripciones y contenido general. Es una sans-serif geométrica moderna que contrasta bien con las tipografías romanas en títulos.

Elementos visuales característicos

- Fondo base: negro puro (#0a0a0a) con gradientes radiales dorados muy sutiles en las esquinas del viewport para dar profundidad.
- Cuadrícula decorativa: pseudoelemento ::before sobre el body con background-image de líneas semitransparentes (60×60px, opacidad 2,5%) que aporta textura sin saturar el diseño.
- Loader animado: pantalla de carga a pantalla completa con el logo y una barra de progreso dorada que desaparece tras 1,5 segundos con transición de opacidad.
- Cursor personalizado: cursor dorado de 20×20px que escala al 200% al pasar sobre elementos interactivos, reemplazando al cursor estándar del navegador.
- Animaciones fadeUp: el contenido entra con una transición de opacidad de 0 a 1 y un desplazamiento vertical de 30px aplicada al cargar cada sección.
- Cards con borde activo: las tarjetas de servicios y productos muestran un borde superior dorado de 2px y un ligero desplazamiento hacia arriba (translateY) al hacer hover.

4.4 Página principal (index.html)

La página principal funciona como una landing page de una sola página con scroll vertical. Está estructurada en seis secciones bien diferenciadas que siguen el patrón clásico de landing page: captar atención → mostrar oferta → generar confianza → impulsar acción.

Navbar

Barra de navegación fija (position: fixed) con el logo de la barbería a la izquierda, el menú de navegación centrado y los botones de acción a la derecha. Implementa un efecto de scroll: al desplazarse más de 80px la navbar cambia de fondo transparente a fondo oscuro opaco con una transición CSS de 0.3 segundos.

Cuando el usuario está autenticado, el botón "Entrar" se transforma dinámicamente en el nombre del usuario enlazado a su panel de perfil. Esto se implementa en main.js al cargar la página:

```
const usuario = JSON.parse(localStorage.getItem("usuario") || "null");
if (usuario) {
  const btnEntrar = document.querySelector("a[href='login.html']");
  if (btnEntrar) {
    btnEntrar.textContent = "👤 " + usuario.nombre;
    btnEntrar.href = "perfil.html";
  }
}
```

Sección Hero

Sección a pantalla completa (100vh) con el logo grande de la barbería, el título principal con la tipografía Cinzel Decorative, una línea decorativa dorada y dos botones de llamada a la acción: "Reservar cita" (enlace a reservas.html) y "Ver servicios" (ancla a #servicios). Incluye un indicador de scroll animado en la parte inferior.

Sección de servicios

Grid de cuatro tarjetas con las fotografías reales de los servicios, nombre, descripción, precio y duración. Los servicios se cargan dinámicamente desde la API al iniciar la página mediante una llamada Fetch. Si el servidor no está disponible, la función falla silenciosamente y se muestran los servicios estáticos del HTML.

La función de carga de servicios implementa el patrón async/await con manejo de errores:

```
async function cargarServicios() {
  try {
    const res = await fetch("http://localhost:3000/api/citas/servicios");
    const data = await res.json();
    // Actualizar cards con datos de la API
  } catch (error) {
    console.log("Servidor no disponible, usando datos estáticos");
  }
}
```

Sección Nosotros

Diseño de dos columnas: imagen del interior de la barbería a la izquierda y contenido descriptivo a la derecha. Incluye una barra de estadísticas con cuatro métricas del negocio: +500 clientes satisfechos, 5 estrellas de valoración media, +10 años de experiencia y 4 servicios premium. En móvil las columnas se apilan verticalmente.

Sección de productos

Grid de cuatro cards de productos con fotografía del producto sobre fondo blanco (para resaltar los detalles del packaging), nombre, descripción breve, precio y botón "+ Carrito". Al

pulsar el botón se llama a la función añadirAlCarrito(producto) que gestiona el estado del carrito en localStorage y muestra una confirmación visual temporal.

Sección CTA de reserva

Sección centrada con un ornamento tipográfico con las letras "ORZ" en grande, el título "Reserva tu Cita" y el botón de llamada a la acción. Es el segundo punto de conversión de la página, situado estratégicamente después de mostrar servicios y productos.

Footer

Pie de página de tres columnas con la descripción de la marca, los enlaces de navegación rápida y la información de contacto completa (dirección, teléfono, email y horario de apertura). En la parte inferior del footer aparece el copyright. En móvil las columnas se apilan.

4.5 Sistema de autenticación

Página de Login (login.html)

Diseño de card centrada verticalmente sobre el fondo oscuro de la aplicación. Incluye el logo de la barbería en la parte superior de la card, los campos de email y contraseña con sus iconos decorativos, el botón de envío y el enlace a la página de registro para usuarios nuevos.

El proceso completo de login en el cliente:

1. El usuario rellena el formulario y pulsa el botón.
2. JavaScript captura el evento submit con un listener en el formulario.
3. Se valida que ambos campos tengan contenido. Si alguno está vacío se muestra un error inmediato sin enviar la petición.
4. Se envía la petición POST a /api/auth/login con Content-Type: application/json y el cuerpo { email, contraseña }.
5. Si el servidor responde con 200 OK se extraen el token y los datos del usuario de la respuesta JSON.
6. Se guardan en localStorage: token JWT bajo la clave "token" y los datos del usuario serializado como JSON bajo la clave "usuario".
7. Se redirige a admin.html si el rol es "admin" o a index.html si es "cliente".
8. Si el servidor responde con error, se muestra el mensaje de error recibido al usuario.

Página de Registro (registro.html)

Formulario con seis campos: nombre y apellidos en una fila de dos columnas, teléfono (opcional), email, contraseña y repetir contraseña. El diseño sigue el mismo estilo visual que el login para mantener coherencia.

Las validaciones del lado del cliente que se ejecutan antes de enviar la petición al servidor:

- Nombre, email, contraseña y repetir contraseña son campos obligatorios.
- La contraseña debe tener un mínimo de 6 caracteres.
- Las dos contraseñas introducidas deben coincidir exactamente.
- El formato del email se valida con el atributo type="email" del input HTML5.

Si todas las validaciones pasan, se envía la petición POST a /api/auth/registro. Si el servidor responde con éxito, se redirige automáticamente al login con un mensaje de "Registro exitoso". Si el email ya existe, el servidor devuelve el error correspondiente que se muestra al usuario.

4.6 Reserva de citas (reservas.html)

La página de reservas implementa un flujo guiado en tres pasos con un indicador de progreso visual en la parte superior. Este diseño step-by-step reduce la carga cognitiva del usuario y facilita el proceso de reserva. La página verifica la autenticación al cargar: si no hay token válido en localStorage redirige inmediatamente al login.

Indicador de progreso

Tres círculos numerados conectados por líneas que representan los pasos del proceso. El paso activo se resalta con el color dorado. Los pasos completados muestran una marca de verificación. Los pasos futuros aparecen en gris.

Paso 1: Selección de servicio

Se realiza una llamada a la API para obtener los servicios disponibles y se renderizan como tarjetas interactivas. Cada tarjeta muestra la imagen del servicio, el nombre, la duración en minutos y el precio. Al hacer clic en una tarjeta:

9. La tarjeta se resalta visualmente con el borde dorado y un check.
10. Se guarda el servicio seleccionado en una variable del estado de la página.
11. Tras un delay de 400ms (para que el usuario vea el feedback visual), la aplicación avanza automáticamente al Paso 2.

Paso 2: Selección de fecha y hora

El usuario selecciona primero la fecha con un input type="date". El atributo min se establece dinámicamente al día actual para impedir la selección de fechas pasadas.

Al cambiar la fecha se ejecuta la función cargarHorariosDisponibles(fecha) que:

12. Realiza una petición GET a /api/citas/horarios-ocupados/{fecha}.

13. El servidor devuelve un array con los objetos de las citas existentes en esa fecha (con hora_inicio y hora_fin de cada una).
14. La función genera los botones de horario de 9:00 a 19:30 en intervalos de 30 minutos.
15. Cada horario se compara con las citas ocupadas para determinar si hay solapamiento.
16. Si la fecha es hoy, los horarios anteriores a la hora actual se deshabilitan.
17. Los horarios ocupados o pasados se muestran con estilo "no disponible" y el cursor no-allowed.

El algoritmo de detección de solapamiento verifica que no exista ninguna cita cuya hora_inicio sea menor que la hora_fin de la cita nueva Y cuya hora_fin sea mayor que la hora_inicio de la cita nueva. Esta comprobación se realiza tanto en el frontend (para la interfaz) como en el backend (para la seguridad).

Existe también un problema especial con las fechas UTC de MySQL. Cuando MySQL devuelve una fecha como "2026-02-28T00:00:00.000Z", JavaScript la interpreta como UTC y al convertirla a fecha local puede quedar un día antes. La solución implementada es añadir el sufijo "T12:00:00" a la cadena de fecha antes de crear el objeto Date, forzando la interpretación al mediodía UTC para evitar el desfase horario:

```
const fechaBruta = cita.fecha;
const fecha = new Date(fechaBruta.substring(0,10) + "T12:00:00");
const fechaFormateada = fecha.toLocaleDateString("es-ES", { ... });
```

Paso 3: Confirmación

Card de resumen con toda la información de la cita: icono del servicio, nombre del servicio, fecha formateada en español con el nombre del día y del mes, hora de inicio, duración y precio. Dos botones: "Volver" para retroceder al paso 2 y "Confirmar cita" para crear la reserva.

Al confirmar se envía la petición POST a /api/citas con el token en el header Authorization y el cuerpo { servicio_id, fecha, hora_inicio }. Si la respuesta es exitosa se muestra un mensaje de confirmación y se redirige al perfil del usuario tras 2 segundos.

4.7 Tienda online y carrito

Gestión del carrito en localStorage

El carrito de compras se implementa como un array de objetos JSON almacenado en localStorage bajo la clave "carrito". Cada objeto del array tiene la estructura:

```
{ id: 1, nombre: "Cera (60ml)", precio: 9.99, cantidad: 2 }
```

Las operaciones del carrito están implementadas en main.js como funciones globales para que sean accesibles desde cualquier página:

- añadirAlCarrito(producto): si el producto ya existe en el carrito (mismo id) incrementa su cantidad en 1. Si no existe, lo añade con cantidad 1. Guarda el carrito actualizado en localStorage y muestra un mensaje de confirmación.
- obtenerCarrito(): lee el carrito de localStorage y lo parsea. Devuelve un array vacío si no existe el elemento.
- guardarCarrito(carrito): serializa el array como JSON y lo guarda en localStorage.
- actualizarContadorCarrito(): cuenta los elementos del carrito y actualiza el badge numérico del icono del carrito en la navbar.

Página del carrito (carrito.html)

Layout de dos columnas: la lista de productos a la izquierda (flex-grow: 1) y el panel de resumen a la derecha (position: sticky, top: 100px). En mobile el panel de resumen se coloca debajo de la lista.

Cada línea del carrito renderiza dinámicamente:

- La imagen del producto (fuente relativa al directorio de imágenes).
- El nombre y el precio unitario del producto.
- Los controles de cantidad: botones "−" y "+" que llaman a modificarCantidad(id, delta) y el número actual en el centro.
- El subtotal calculado como precio × cantidad, formateado con toFixed(2).
- El botón de eliminar con icono de papelera.

El panel de resumen muestra el subtotal, los gastos de envío (calculados en base al total) y el total final con el IVA incluido. El botón "Finalizar compra" verifica que el usuario esté autenticado antes de procesar:

```
async function finalizarCompra() {
  const token = localStorage.getItem("token");
  if (!token) {
    alert("Debes iniciar sesión para finalizar la compra");
    window.location.href = "login.html";
    return;
  }
  // Enviar pedido al backend...
}
```

4.8 Panel de usuario (perfil.html)

Panel personal del cliente con diseño de card centrada. En la cabecera se muestra el avatar genérico, el nombre completo y el email del usuario (obtenidos de localStorage). El botón de cierre de sesión en la esquina superior derecha llama a logout() que elimina el token y los datos de localStorage y redirige a index.

Pestañas de navegación

Dos pestañas: "Mis Citas" y "Mis Pedidos". Al hacer clic en una pestaña se muestra el contenido correspondiente ocultando el otro. La pestaña activa se resalta con el color

dorado. El contenido de ambas pestañas se carga al abrir la página mediante llamadas paralelas a la API con Promise.all() para optimizar los tiempos de carga.

Pestaña Mis Citas

Realiza una petición GET a /api/citas/mis-citas con el token en el header Authorization. El servidor devuelve las citas del usuario autenticado con todos sus datos incluyendo el nombre del servicio (obtenido mediante JOIN con la tabla servicios).

Cada cita se muestra en una card con:

- Nombre del servicio en el título.
- Fecha formateada en español (con el truco T12:00:00 para evitar el desfase UTC).
- Horario (hora_inicio – hora_fin).
- Precio cobrado formateado con dos decimales y símbolo €.
- Badge de estado con codificación visual por colores: dorado (pendiente), verde (confirmada), azul (completada), rojo (cancelada).
- Botón "Cancelar" visible únicamente para citas en estado pendiente o confirmada.

Pestaña Mis Pedidos

Realiza una petición GET a /api/tienda/mis-pedidos con el token. Muestra cada pedido con el número de pedido, la fecha de creación formateada, el total y el estado. La respuesta de la API incluye también las líneas del pedido agrupadas para mostrar el detalle de los productos comprados.

4.9 Panel de administración (admin.html)

El panel de administración es la herramienta de gestión del negocio. Verifica al cargar que el usuario tenga el token válido Y que su rol sea "admin". Si alguna de las dos condiciones no se cumple, muestra un alert de "Acceso restringido" y redirige a index.html.

```
const token = localStorage.getItem("token");
const usuario = JSON.parse(localStorage.getItem("usuario") || "null");
if (!token || !usuario || usuario.rol !== "admin") {
  alert("Acceso restringido. Solo administradores.");
  window.location.href = "index.html";
}
```

Bloque de estadísticas

Al cargar la página se realizan tres peticiones en paralelo con Promise.all() para obtener el total de citas, el total de pedidos y el número de productos activos. Los resultados se muestran en tres tarjetas de estadísticas en la parte superior del panel.

Gestión de citas

Tabla con paginación que muestra todas las citas del sistema. Las columnas son: ID, cliente (nombre + apellidos), servicio, fecha, hora de inicio, precio cobrado y estado.

El filtro por estado se implementa con un select en la parte superior de la tabla que oculta/muestra las filas según el estado seleccionado usando display: none/table-row en el CSS.

El selector de estado en cada fila de la tabla dispara la función cambiarEstadoCita(id, nuevoEstado) al cambiar. Esta función realiza una petición PUT a /api/citas/{id}/estado con el nuevo estado en el cuerpo. Al recibir respuesta 200 actualiza visualmente el badge de estado de esa fila sin recargar toda la tabla.

Gestión de pedidos

Tabla que muestra todos los pedidos del sistema con columnas para ID, nombre del cliente, email, fecha de creación, total y estado. La información del cliente se obtiene mediante JOIN en el servidor entre las tablas pedidos y usuarios.

Gestión de productos

Tabla de productos con columnas para ID, nombre, descripción (truncada a 50 caracteres), precio, stock y acciones. Los botones de acción en cada fila son "Editar" (abre el modal con datos prellenados) y "Borrar" (elimina con confirmación previa).

Modal de producto

Formulario modal (position: fixed, z-index elevado, overlay semitransparente) que se usa tanto para crear nuevos productos como para editar los existentes. La lógica de "modo creación" vs "modo edición" se gestiona con una variable modoEdicion y el id del producto actual:

```
let modoEdicion = false;
let productoActualId = null;

function abrirModalEdicion(producto) {
  modoEdicion = true;
  productoActualId = producto.id;
  document.getElementById("modal-nombre").value = producto.nombre;
  // ... rellenar demás campos
  document.getElementById("modal-producto").style.display = "flex";
}
```

5. Desarrollo Backend

5.1 Tecnologías y dependencias

El backend está construido sobre Node.js, un entorno de ejecución JavaScript en el servidor basado en el motor V8 de Chrome. La elección de Node.js se justifica por su modelo de I/O no bloqueante y asíncrono, que lo hace especialmente eficiente para aplicaciones web con muchas operaciones de entrada/salida como consultas a la base de datos y llamadas entre servicios.

Paquete npm	Versión	Propósito y justificación
express	^4.18	Framework web minimalista para Node.js. Proporciona el router, el middleware y los helpers para crear la API REST.
mysql2	^3.6	Driver MySQL moderno con soporte nativo de promesas y prepared statements para prevenir SQL injection.
bcrypt	^5.1	Implementación del algoritmo bcrypt para hashing de contraseñas. Incluye salt automático y es resistente a ataques de fuerza bruta.
jsonwebtoken	^9.0	Generación y verificación de JSON Web Tokens (JWT) según el estándar RFC 7519.
dotenv	^16.3	Carga de variables de entorno desde archivo .env para separar la configuración del código.
cors	^2.8	Middleware que configura las cabeceras CORS para permitir peticiones del frontend desde diferente origen.

5.2 Arquitectura del servidor

El servidor sigue una arquitectura de tres capas que separa claramente las responsabilidades y facilita el mantenimiento y la escalabilidad del código:

- Capa de presentación (routes/): define las URLs disponibles de la API, aplica el middleware correspondiente a cada ruta y delega la ejecución a los controladores.
- Capa de lógica de negocio (controllers/): implementa la lógica de cada operación: valida los datos recibidos, interactúa con la base de datos y prepara la respuesta.
- Capa de datos (config/db.js): gestiona el pool de conexiones a MySQL y expone una función para ejecutar consultas SQL de forma asíncrona.

El archivo index.js es el punto de entrada del servidor. Configura Express con los middlewares globales (cors, express.json para parsear el cuerpo de las peticiones), registra los routers de cada módulo y arranca el servidor en el puerto definido en el .env:

```
const express = require("express");
```

```
const cors    = require("cors");
require("dotenv").config();

const authRoutes   = require("./routes/authRoutes");
const citasRoutes  = require("./routes/citasRoutes");
const tiendaRoutes = require("./routes/tiendaRoutes");

const app = express();
app.use(cors());
app.use(express.json());

app.use("/api/auth",   authRoutes);
app.use("/api/citas",  citasRoutes);
app.use("/api/tienda", tiendaRoutes);

app.listen(process.env.PORT, () => {
  console.log(`🚀 Servidor en http://localhost:${process.env.PORT}`);
});
```

5.3 Configuración y variables de entorno

La configuración del servidor se gestiona mediante variables de entorno almacenadas en el archivo .env en la raíz del directorio backend. Este archivo contiene las credenciales de la base de datos y la clave secreta JWT. El archivo nunca debe incluirse en el control de versiones y se excluye en el .gitignore.

Variable	Ejemplo de valor	Descripción
PORT	3000	Puerto en el que escucha el servidor HTTP
DB_HOST	localhost	Dirección del servidor MySQL
DB_USER	root	Usuario de la base de datos MySQL
DB_PASSWORD	mi_password	Contraseña del usuario de MySQL
DB_NAME	barberia_db	Nombre de la base de datos a usar
JWT_SECRET	s3cr3t_k3y_muy_larga	Clave para firmar y verificar los tokens JWT. Debe ser larga y aleatoria.

La conexión a MySQL se implementa mediante un pool de conexiones que reutiliza conexiones abiertas, evitando el coste de crear una nueva conexión para cada petición:

```
const mysql = require("mysql2/promise");

const pool = mysql.createPool({
  host:     process.env.DB_HOST,
  user:     process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
```

```
});

module.exports = pool;
```

5.4 Módulo de autenticación

POST /api/auth/registro

Este endpoint crea una nueva cuenta de usuario. Recibe en el cuerpo de la petición los campos nombre, apellidos, telefono, email y contraseña. La implementación del controlador:

18. Verifica que los campos obligatorios (nombre, email, contraseña) estén presentes. Si falta alguno devuelve 400 Bad Request.
19. Ejecuta una consulta SELECT para comprobar si ya existe un usuario con ese email. Si existe devuelve 409 Conflict.
20. Genera el hash de la contraseña con bcrypt.hash(contraseña, 10). El factor de coste 10 es el recomendado actualmente por la OWASP para bcrypt.
21. Inserta el nuevo usuario en la base de datos con la contraseña hasheada y el rol "cliente" por defecto.
22. Devuelve 201 Created con el ID del usuario creado.

POST /api/auth/login

Verifica las credenciales y emite un token JWT. El controlador:

23. Busca al usuario por email. Si no existe devuelve 401 (sin especificar si es el email o la contraseña erróneos, por seguridad).
24. Compara la contraseña recibida con el hash almacenado usando bcrypt.compare(). Si no coincide devuelve 401.
25. Genera el token JWT: jwt.sign({ id: usuario.id, rol: usuario.rol }, process.env.JWT_SECRET, { expiresIn: "8h" }).
26. Devuelve 200 OK con el token y los datos públicos del usuario (id, nombre, apellidos, email, rol).

> ⚠ *Por seguridad, el servidor devuelve el mismo mensaje de error tanto si el email no existe como si la contraseña es incorrecta, para no revelar qué usuarios están registrados en el sistema.*

5.5 Módulo de reservas y citas

GET /api/citas/servicios

Devuelve todos los servicios disponibles. Sin autenticación requerida. Ejecuta SELECT * FROM servicios ORDER BY precio ASC y devuelve el array de servicios.

GET /api/citas/horarios-ocupados/:fecha

Endpoint fundamental para el control de disponibilidad. Recibe la fecha como parámetro de URL (formato YYYY-MM-DD) y devuelve todas las citas registradas para esa fecha que no estén canceladas:

```
SELECT hora_inicio, hora_fin FROM citas
WHERE fecha = ? AND estado != "cancelada"
```

El frontend utiliza esta lista para deshabilitar los slots de tiempo que se solaparían con alguna cita existente.

POST /api/citas

Crea una nueva reserva. Requiere autenticación (rol cliente o admin). El controlador recibe servicio_id, fecha y hora_inicio:

27. Obtiene los datos del servicio (precio y duración) de la base de datos.
28. Calcula la hora_fin sumando la duración del servicio a hora_inicio.
29. Ejecuta una consulta de solapamiento para verificar que no exista ninguna cita en ese rango horario:
    ```
    SELECT id FROM citas WHERE fecha = ? AND estado != "cancelada"
    AND hora_inicio < ? AND hora_fin > ?
    ```
30. Si hay solapamiento devuelve 409 Conflict con el mensaje "Horario no disponible".
31. Si no hay solapamiento, inserta la cita con el precio_cobrado obtenido del servicio.
32. Devuelve 201 Created con el ID de la nueva cita.

GET /api/citas/mis-citas

Devuelve las citas del usuario autenticado. Realiza un JOIN con la tabla servicios para incluir el nombre del servicio en cada cita, ordenadas por fecha descendente:

```
SELECT c.*, s.nombre as servicio, s.duracion
FROM citas c
JOIN servicios s ON c.servicio_id = s.id
WHERE c.usuario_id = ?
ORDER BY c.fecha DESC, c.hora_inicio DESC
```

PUT /api/citas/cancelar/:id

Permite a un cliente cancelar su propia cita. El middleware verifica el token y el controlador verifica además que la cita pertenezca al usuario autenticado y que su estado sea "pendiente" o "confirmada". Si alguna condición no se cumple devuelve el error apropiado.

GET /api/citas (Admin)

Devuelve todas las citas del sistema con datos del cliente. Requiere rol "admin". Realiza un JOIN triple entre citas, usuarios y servicios:

```
SELECT c.*, u.nombre, u.apellidos, u.email, s.nombre as servicio
FROM citas c
JOIN usuarios u ON c.usuario_id = u.id
JOIN servicios s ON c.servicio_id = s.id
ORDER BY c.fecha DESC, c.hora_inicio DESC
```

PUT /api/citas/:id/estado (Admin)

Cambia el estado de una cita. Recibe el nuevo estado en el cuerpo de la petición. Valida que el estado sea uno de los valores permitidos del ENUM antes de ejecutar el UPDATE.

5.6 Módulo de tienda

Método	Ruta	Auth	Descripción
GET	/api/tienda/productos	Pública	Catálogo completo de productos activos
POST	/api/tienda/pedidos	Cliente	Crea pedido con sus líneas en transacción
GET	/api/tienda/mis-pedidos	Cliente	Pedidos del usuario autenticado con líneas
GET	/api/tienda/pedidos	Admin	Todos los pedidos con datos del cliente
POST	/api/tienda/productos	Admin	Crea un nuevo producto en el catálogo
PUT	/api/tienda/productos/:id	Admin	Actualiza datos de un producto existente
DELETE	/api/tienda/productos/:id	Admin	Elimina un producto del catálogo

POST /api/tienda/pedidos – Detalle de implementación

Este endpoint es el más complejo del módulo de tienda porque debe crear el pedido y todas sus líneas de forma atómica usando una transacción MySQL:

33. Se obtiene una conexión del pool.
34. Se inicia la transacción con connection.beginTransaction().
35. Se calcula el total sumando precio × cantidad de cada línea del array recibido.
36. Se inserta el registro en la tabla pedidos (usuario_id, total, estado "pendiente").
37. Se obtiene el ID del pedido recién creado con result.insertId.
38. Para cada línea del pedido, se inserta un registro en pedidos_linea con el pedido_id, producto_id, cantidad y precio_unitario.

39. Se confirma la transacción con connection.commit().
40. Si ocurre cualquier error en algún paso, se ejecuta connection.rollback() para deshacer todos los cambios y se devuelve error 500.

5.7 Seguridad: JWT y bcrypt

JSON Web Tokens (JWT)

Los JWT son un estándar abierto (RFC 7519) que define una forma compacta y autónoma de transmitir información de forma segura entre partes como un objeto JSON. Un token JWT consta de tres partes separadas por puntos:

- Header: tipo de token (JWT) y algoritmo de firma (HS256).
- Payload: datos del usuario (id y rol) más las fechas de emisión (iat) y expiración (exp).
- Signature: firma HMAC-SHA256 del header y el payload usando la clave secreta del servidor.

La firma garantiza que el contenido del token no ha sido modificado. Sin embargo, el payload no está cifrado (solo codificado en base64), por lo que nunca debe incluir información sensible como contraseñas.

El ciclo de vida del token en la aplicación:

41. El servidor genera el token en el login con jwt.sign({ id, rol }, secret, { expiresIn: "8h" }).
42. El cliente almacena el token en localStorage.
43. El cliente envía el token en cada petición privada: Authorization: Bearer {token}.
44. El middleware del servidor verifica el token con jwt.verify(token, secret).
45. Si el token es válido y no ha expirado, jwt.verify devuelve el payload decodificado.
46. El middleware añade el payload al objeto request (req.usuario) para que el controlador acceda a los datos del usuario sin nueva consulta a la BD.
47. Si el token ha expirado o es inválido, jwt.verify lanza una excepción y el middleware devuelve 401.

bcrypt y el hash de contraseñas

bcrypt es un algoritmo de hash diseñado específicamente para contraseñas. Sus características lo hacen ideal para este propósito:

- Incorpora un salt aleatorio automáticamente, lo que hace que dos contraseñas idénticas produzcan hashes distintos y previene los ataques de tablas arco iris.
- Es computacionalmente costoso de forma intencionada. El factor de coste (10 en este proyecto) determina cuántas iteraciones se ejecutan (2^10 = 1024). Esto hace que calcular millones de hashes para un ataque de fuerza bruta sea inviable.

- El hash resultante de 60 caracteres incluye el salt, por lo que bcrypt.compare puede verificar cualquier contraseña contra cualquier hash sin necesidad de almacenar el salt por separado.

Para actualizar la contraseña del administrador desde la terminal de Node.js se puede usar:

```
node -e "const
b=require('bcrypt');b.hash('nueva_pass',10).then(h=>console.log(h));"
```

5.8 Middleware de protección de rutas

El middleware de autenticación es una función de Express que se ejecuta antes del controlador en las rutas protegidas. Su implementación completa:

```
const jwt = require("jsonwebtoken");

function verificarToken(req, res, next) {
  const authHeader = req.headers["authorization"];
  if (!authHeader) return res.status(401).json({ error: "Token requerido"
});

  const token = authHeader.split(" ")[1]; // "Bearer TOKEN"
  if (!token) return res.status(401).json({ error: "Formato de token
inválido" });

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.usuario = decoded;
    next();
  } catch (error) {
    return res.status(401).json({ error: "Token inválido o expirado" });
  }
}

function soloAdmin(req, res, next) {
  if (req.usuario.rol !== "admin") {
    return res.status(403).json({ error: "Acceso denegado" });
  }
  next();
}
```

Las rutas del router usan estos middlewares encadenados:

```
// Solo autenticado
router.get("/mis-citas", verificarToken, getMisCitas);

// Solo admin
router.get("/", verificarToken, soloAdmin, getAllCitas);
```

6. Manual de Usuario

6.1 Requisitos del sistema

Software	Versión mínima	Propósito
Node.js	v18.0 LTS	Ejecutar el servidor backend de la aplicación
MySQL Server	8.0	Base de datos relacional de la aplicación
MySQL Workbench	Cualquiera	Importar el esquema SQL e inspeccionar los datos
Visual Studio Code	Cualquiera	Editor recomendado para abrir y ejecutar el proyecto
Live Server (ext.)	Cualquiera	Servir el frontend con recarga automática
npm	v8.0+	Gestor de paquetes para instalar dependencias de Node.js
Chrome / Firefox / Edge	v90+	Navegar por la aplicación

6.2 Instalación paso a paso

1. Configurar la base de datos

48. Abrir MySQL Workbench y conectarse al servidor MySQL local (generalmente localhost:3306).
49. En el menú File → Open SQL Script, abrir el archivo "barberia_schema.sql" incluido en la carpeta del proyecto.
50. Ejecutar el script completo pulsando el botón del rayo ⚡ o con el atajo Ctrl+Shift+Enter.
51. Verificar en el panel Navigator que se ha creado la base de datos "barberia_db" con sus 6 tablas.
52. Expandir la base de datos y verificar que las tablas tienen datos: ejecutar SELECT COUNT(*) FROM usuarios y esperar al menos 3 resultados.

2. Configurar las variables de entorno

53. Navegar al directorio "backend" del proyecto.
54. Verificar que existe el archivo ".env". Si no existe, crear uno nuevo.
55. Comprobar que las variables DB_HOST, DB_USER, DB_PASSWORD y DB_NAME coinciden con la configuración de MySQL del equipo.
56. Verificar que JWT_SECRET tiene un valor largo y aleatorio para mayor seguridad.

3. Instalar dependencias y arrancar el servidor

57. Abrir Visual Studio Code en la carpeta raíz del proyecto.
58. Abrir la terminal integrada con Ctrl + Ñ.

59. Ejecutar: cd backend
60. Ejecutar: npm install (instalará todas las dependencias listadas en package.json).
61. Ejecutar: node index.js
62. Verificar que aparecen los mensajes: "🚀 Servidor corriendo en http://localhost:3000" y "✅ Conectado a MySQL correctamente".

> ⚠ *El servidor backend debe estar en ejecución durante toda la sesión de uso. Si se cierra el terminal o el servidor se detiene, la aplicación no podrá conectar con la base de datos y mostrará errores.*

4. Abrir el frontend con Live Server

63. En el explorador de archivos de VS Code, expandir la carpeta "frontend/pages".
64. Hacer clic derecho sobre "index.html".
65. Seleccionar "Open with Live Server".
66. El navegador se abrirá automáticamente con la URL http://127.0.0.1:5500/frontend/pages/index.html.
67. Verificar que la página principal carga correctamente con el loader animado.

> ⚠ *Es imprescindible usar Live Server o un servidor HTTP para abrir el frontend. Abrir los archivos HTML directamente (con file://) causará errores de CORS en las peticiones a la API.*

6.3 Guía del cliente

Crear una cuenta

Para reservar citas o comprar productos es necesario registrarse en la aplicación. El proceso de registro está diseñado para ser rápido y sencillo:

68. En la barra de navegación superior, hacer clic en el botón "Registro".
69. En la página de registro, rellenar los campos del formulario:
    - Nombre (obligatorio): nombre de pila del usuario.
    - Apellidos (opcional pero recomendado).
    - Teléfono (opcional).
    - Email (obligatorio): será el identificador de acceso al sistema.
    - Contraseña (obligatorio): mínimo 6 caracteres.
    - Repetir contraseña (obligatorio): debe coincidir con la anterior.
70. Pulsar el botón "Crear cuenta".
71. Si el registro es correcto, el sistema mostrará un mensaje de éxito y redirigirá al login.

Iniciar y cerrar sesión

Para iniciar sesión hacer clic en "Entrar" en la navbar, introducir el email y la contraseña y pulsar "Entrar al Studio". El nombre del usuario aparecerá en la navbar como confirmación de que la sesión está activa.

Para cerrar sesión, acceder al panel de perfil haciendo clic en el nombre de usuario en la navbar y pulsar el botón "Cerrar sesión" en la esquina superior de la página.

Reservar una cita – guía paso a paso

El proceso de reserva se completa en tres pasos guiados. El tiempo estimado para realizar una reserva completa es de 1 a 2 minutos:

Paso 1 – Elegir el servicio:

72. Desde la página principal hacer clic en el botón "Reservar cita" de la sección hero o en "Reservar ahora" de la sección de reservas.
73. Si no has iniciado sesión, el sistema te redirigirá al login antes de continuar.
74. En la página de reservas se muestran los cuatro servicios disponibles con su fotografía, nombre, duración y precio.
75. Hacer clic sobre la tarjeta del servicio que deseas reservar. La tarjeta se resaltará en dorado y la aplicación avanzará automáticamente al paso 2.

Paso 2 – Elegir fecha y hora:

76. Hacer clic en el campo de fecha y seleccionar el día deseado en el calendario. No es posible seleccionar fechas anteriores al día de hoy.
77. Al seleccionar la fecha, aparecerán los botones de horario. Los horarios en color oscuro con texto gris están ocupados o son horas pasadas y no se pueden seleccionar. Los horarios en dorado están disponibles.
78. Hacer clic en el horario disponible que prefieras.
79. Pulsar el botón "Siguiente" para avanzar al paso 3.

Paso 3 – Confirmar la reserva:

80. Revisar el resumen de la cita: servicio, fecha completa en español, hora, duración y precio.
81. Si algo no es correcto, pulsar "Volver" para modificar la fecha o la hora, o pulsar el indicador del Paso 1 para cambiar el servicio.
82. Si todo es correcto, pulsar "Confirmar cita".
83. El sistema creará la cita y mostrará un mensaje de confirmación. Serás redirigido a tu panel de perfil donde podrás ver la cita en "Mis Citas".

Comprar productos

84. En la sección "Tienda" de la página principal, ver los productos disponibles.
85. Pulsar el botón "+ Carrito" en el producto deseado. Aparecerá un mensaje de confirmación temporal.

86. Repetir el proceso para añadir más productos. El icono del carrito en la navbar mostrará el número de artículos.
87. Hacer clic en el icono "🛒 Carrito" de la navbar para acceder a la página del carrito.
88. Revisar los productos: si quieres más unidades de un producto pulsa el botón "+", para reducir pulsa "−". Si la cantidad llega a 0 el producto se elimina del carrito.
89. Verificar el total en el panel de resumen a la derecha.
90. Pulsar "Finalizar compra". Si no has iniciado sesión el sistema te redirigirá al login.
91. Confirmar la compra. El pedido quedará registrado y el carrito se vaciará.

Consultar citas y pedidos

92. Hacer clic en tu nombre en la barra de navegación para acceder al panel de perfil.
93. En la pestaña "Mis Citas" verás todas tus reservas ordenadas por fecha. El estado de cada cita aparece en un badge de color: dorado (pendiente), verde (confirmada), azul (completada), rojo (cancelada).
94. Para cancelar una cita pendiente o confirmada, pulsar el botón "Cancelar" en su tarjeta.
95. En la pestaña "Mis Pedidos" verás el historial de tus compras con el detalle de los productos de cada pedido.

6.4 Guía del administrador

Acceso al panel de administración

Iniciar sesión con las credenciales de administrador:

Email	admin@barberia.com
Contraseña	admin123

Al hacer login con estas credenciales, el sistema detectará el rol "admin" y redirigirá automáticamente al panel de administración (admin.html). Si intentas acceder directamente a la URL admin.html sin autenticación o con rol cliente, serás redirigido a la página principal.

Vista general y estadísticas

Al cargar el panel de administración se muestran tres tarjetas de estadísticas en la parte superior:

- Total de citas: número total de citas registradas en el sistema, de todos los estados.
- Total de pedidos: número total de pedidos realizados por todos los clientes.
- Productos activos: número de productos disponibles actualmente en el catálogo.

Gestión de citas

La pestaña "Citas" muestra una tabla con todas las reservas del sistema. Instrucciones de uso:

96. Para filtrar las citas por estado, usar el selector desplegable situado en la esquina superior derecha de la tabla. Las opciones son: Todas, Pendiente, Confirmada, Completada y Cancelada.
97. Para cambiar el estado de una cita, localizar la fila de la cita en la tabla y hacer clic en el selector de la columna "Estado".
98. Seleccionar el nuevo estado del menú desplegable. El cambio se aplica inmediatamente sin necesidad de guardar: el sistema envía la actualización al servidor en el momento de la selección.
99. El badge de estado en esa fila cambia de color automáticamente para reflejar el nuevo estado.

Ciclo de vida habitual de una cita:

- Pendiente → Confirmada: cuando el barbero confirma que tiene disponibilidad.
- Confirmada → Completada: cuando la cita se ha realizado satisfactoriamente.
- Pendiente / Confirmada → Cancelada: cuando el cliente o el negocio cancela.

Gestión de pedidos

La pestaña "Pedidos" muestra todos los pedidos realizados por los clientes. La tabla incluye el ID del pedido, el nombre completo del cliente, su email, la fecha de realización formateada y el total en euros. En esta versión del proyecto los pedidos son de solo consulta: no se pueden modificar desde la interfaz.

Gestión de productos – Crear nuevo producto

100. Hacer clic en el botón "+ Nuevo producto" situado sobre la tabla de productos.
101. Se abrirá el formulario modal de creación. Rellenar los campos:
    - Nombre (obligatorio): nombre del producto tal como aparecerá en la tienda.
    - Descripción (opcional): texto descriptivo del producto.
    - Precio (obligatorio): precio de venta en euros. Usar punto como separador decimal.
    - Stock: unidades disponibles en inventario.
102. Pulsar "Guardar". El nuevo producto aparecerá inmediatamente en la tabla y en el catálogo de la tienda.

Gestión de productos – Editar producto existente

103. En la tabla de productos, localizar el producto que deseas modificar.
104. Pulsar el botón "Editar" en la fila del producto.
105. El formulario modal se abrirá con todos los campos prellenados con los datos actuales del producto.
106. Modificar los campos que sea necesario.
107. Pulsar "Guardar". Los cambios se aplican inmediatamente y la tabla se actualiza.

Gestión de productos – Eliminar producto

108. En la tabla de productos, localizar el producto que deseas eliminar.
109. Pulsar el botón "Borrar" en la fila del producto.
110. El navegador mostrará un diálogo de confirmación: "¿Estás seguro de que quieres eliminar este producto?".
111. Pulsar "Aceptar" para confirmar la eliminación.
112. El producto desaparecerá de la tabla y del catálogo de la tienda de forma inmediata.

⚠ *La eliminación de un producto es permanente. Si el producto tiene pedidos asociados en la base de datos, el servidor devolverá un error y no permitirá su eliminación para mantener la integridad de los datos históricos.*

7. Conclusiones

7.1 Resultados obtenidos

El proyecto Barber O.R.Z Studio ha alcanzado todos los objetivos planteados al inicio del desarrollo. Se ha construido una aplicación web fullstack completamente funcional que cubre el ciclo de vida completo de un proyecto de software profesional: desde el análisis de requisitos y el diseño de la base de datos hasta el desarrollo, las pruebas y la documentación.

Los resultados concretos y cuantificables del proyecto son:

- Base de datos MySQL con 6 tablas, 5 claves foráneas y datos iniciales de prueba.
- Backend Node.js/Express con 18 endpoints REST organizados en 3 módulos (auth, citas, tienda).
- Frontend con 7 páginas HTML, 6 hojas de estilos CSS y 6 scripts JavaScript.
- Sistema de autenticación JWT completamente funcional con diferenciación de roles.
- Sistema de reservas online con control de disponibilidad en tiempo real y prevención de solapamientos.
- Tienda online con carrito persistente y proceso de pedido con transacciones SQL.
- Panel de usuario con historial de citas y pedidos.
- Panel de administración con CRUD de productos y gestión de estados de citas.
- Diseño visual profesional con identidad corporativa propia, responsivo en todos los dispositivos.

7.2 Dificultades y soluciones

El desarrollo del proyecto presentó varias dificultades técnicas que requirieron investigación y creatividad para resolverse. A continuación se documentan las más significativas:

Dificultad 1: Desfase de zona horaria en fechas

Problema: MySQL almacena los campos DATE en UTC. Al serializar una fecha como "2026-02-28" en formato ISO y deserializarla en JavaScript con new Date("2026-02-28"), el navegador la interpreta como medianoche UTC y la convierte a la hora local, resultando en "2026-02-27" en zonas horarias positivas.

Solución: añadir el sufijo "T12:00:00" a la cadena de fecha antes de crear el objeto Date para forzar la interpretación al mediodía UTC, garantizando que la fecha local siempre sea correcta independientemente de la zona horaria del usuario.

Dificultad 2: Nombres de campos de la API no coincidentes

Problema: el frontend asumía que la API devolvía campos con nombres como "precio" y "servicio_nombre", pero el backend los devolvía como "precio_cobrado" y "servicio" (alias del JOIN). Esto causaba que los valores aparecieran como undefined en la interfaz.

Solución: usar las herramientas de desarrollo del navegador (pestaña Network) para inspeccionar la respuesta real de la API y ajustar el código del frontend para usar los nombres exactos devueltos por el servidor.

Dificultad 3: Código duplicado entre HTML inline y archivos .js

Problema: en varias páginas existía código JavaScript tanto en bloques script inline del HTML como en los archivos .js externos. Esto causaba errores de "Cannot redeclare block-scoped variable" y comportamientos inesperados.

Solución: consolidar todo el código JavaScript en los archivos .js externos y eliminar completamente los bloques script del HTML, dejando solo la etiqueta de importación del archivo externo.

Dificultad 4: Contraseña del administrador en bcrypt

Problema: el usuario admin@barberia.com no podía hacer login porque la contraseña en la base de datos estaba en texto plano, pero bcrypt.compare() la comparaba con un hash, resultando siempre en false.

Solución: generar el hash correcto desde la terminal de Node.js y actualizar el campo contraseña en la base de datos mediante MySQL Workbench con una sentencia UPDATE.

7.3 Competencias adquiridas

El desarrollo de este proyecto ha permitido consolidar y ampliar un conjunto amplio de competencias técnicas y transversales:

Competencias técnicas

- Diseño de bases de datos relacionales: modelo E-R, normalización hasta 3FN, claves foráneas, integridad referencial, tipos de datos MySQL.
- Desarrollo de APIs REST con Node.js: enrutado modular, middlewares, manejo de errores HTTP, transacciones SQL.
- Seguridad web: autenticación JWT, hashing de contraseñas con bcrypt, protección de rutas, control de roles, CORS.
- Desarrollo frontend sin frameworks: JavaScript ES6+ (async/await, Fetch API, módulos), manipulación del DOM, eventos, localStorage.
- CSS avanzado: variables custom, flexbox, CSS grid, animaciones, pseudo-elementos, media queries para diseño responsivo.
- Depuración: uso de DevTools del navegador (Network, Console, Sources), interpretación de errores HTTP y de JavaScript.

Competencias transversales

- Metodología de desarrollo: planificación por fases, separación de responsabilidades, código modular y mantenible.
- Resolución de problemas: identificación de la causa raíz de los errores y búsqueda sistemática de soluciones.

- Documentación técnica: redacción de especificaciones de requisitos, descripción de casos de uso, manuales de usuario.

7.4 Mejoras futuras

Si el proyecto continuara su desarrollo, las mejoras más relevantes que se implementarían son:

Mejoras de funcionalidad

- Integración de pasarela de pago real: implementar Stripe o PayPal para que los clientes puedan pagar sus pedidos online y abonar una señal al reservar la cita.
- Notificaciones automáticas: enviar emails de confirmación al reservar una cita o realizar un pedido, y recordatorios automáticos 24 horas antes de la cita mediante un servicio como Nodemailer y SendGrid.
- Sistema de valoraciones: permitir a los clientes puntuar los servicios recibidos (1-5 estrellas) y dejar comentarios. Mostrar la puntuación media en las cards de servicios.
- Gestión de múltiples barberos: si la barbería cuenta con varios profesionales, asignar cada cita a un barbero específico con su propia agenda independiente.
- Vista de calendario en el admin: mostrar las citas del día y de la semana en un calendario visual en lugar de una tabla para facilitar la gestión del negocio.
- Búsqueda y filtrado en la tienda: añadir un campo de búsqueda de productos y filtros por categoría o rango de precio.

Mejoras técnicas

- Migración a React o Vue.js: refactorizar el frontend como una SPA (Single Page Application) utilizando la misma API REST ya desarrollada, mejorando la experiencia de usuario.
- Implementación de refresh tokens: tokens de renovación automática para que el usuario no tenga que hacer login cada 8 horas.
- Tests automatizados: tests unitarios para los controladores del backend con Jest y tests de extremo a extremo con Cypress o Playwright.
- Dockerización: contenedores Docker para el servidor Node.js y la base de datos MySQL, facilitando el despliegue en cualquier entorno.
- Despliegue en producción: configurar la aplicación en un servicio cloud como Railway, Render o un VPS, con dominio propio y certificado SSL.

7.5 Valoración personal

El desarrollo de Barber O.R.Z Studio ha sido la experiencia de aprendizaje más completa y exigente del ciclo formativo. Por primera vez, todas las capas de una aplicación real han tenido que funcionar juntas de forma coordinada: la base de datos respondiendo correctamente a las consultas, el servidor procesando las peticiones de forma segura, el frontend comunicándose con la API y reflejando los datos en tiempo real en la interfaz.

La decisión de desarrollar el frontend sin frameworks fue deliberada y, en retrospectiva, muy acertada desde el punto de vista del aprendizaje. Haber implementado el carrito de compras, el proceso de reservas, el sistema de autenticación y los paneles de usuario y administrador con JavaScript puro ha proporcionado una comprensión profunda de cómo funcionan estas funcionalidades a nivel fundamental, conocimiento que resulta invaluable al trabajar posteriormente con frameworks como React o Vue.

El proceso de depuración fue quizás la parte más formativa del proyecto. Los problemas con las fechas UTC, los nombres de campos de la API, el código duplicado y la contraseña del administrador en bcrypt requirieron diagnosticar la causa raíz de cada error, investigar la solución correcta y aplicarla de forma que no rompiera otras partes del sistema. Esta habilidad de debugging sistemático es fundamental en el trabajo diario de cualquier desarrollador web.

En definitiva, Barber O.R.Z Studio demuestra que los conocimientos adquiridos durante el Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Web son suficientes para construir una aplicación web profesional y completamente funcional que podría implantarse en un negocio real. El proyecto ha generado la confianza y la motivación necesarias para afrontar proyectos de mayor envergadura en la carrera profesional.

Bibliografía y Referencias

Documentación oficial

- MDN Web Docs. Referencia completa de HTML, CSS y JavaScript. Mozilla Developer Network. Disponible en: https://developer.mozilla.org
- Node.js Foundation. Node.js v20.x Documentation. Disponible en: https://nodejs.org/docs/latest-v20.x/api/
- OpenJS Foundation. Express.js 4.x API Reference. Disponible en: https://expressjs.com/en/4x/api.html
- Oracle Corporation. MySQL 8.0 Reference Manual. Disponible en: https://dev.mysql.com/doc/refman/8.0/en/
- Auth0. Introduction to JSON Web Tokens (JWT). Disponible en: https://jwt.io/introduction
- OWASP Foundation. Password Storage Cheat Sheet. Disponible en: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Paquetes npm y repositorios

- npm Registry. express (v4.x). Disponible en: https://www.npmjs.com/package/express
- npm Registry. mysql2 (v3.x). Disponible en: https://www.npmjs.com/package/mysql2
- npm Registry. bcrypt (v5.x). Disponible en: https://www.npmjs.com/package/bcrypt
- npm Registry. jsonwebtoken (v9.x). Disponible en: https://www.npmjs.com/package/jsonwebtoken
- npm Registry. dotenv (v16.x). Disponible en: https://www.npmjs.com/package/dotenv
- npm Registry. cors (v2.x). Disponible en: https://www.npmjs.com/package/cors

Recursos educativos en vídeo

- Traversy Media. Node.js API Masterclass With Express & MongoDB. Disponible en: https://www.udemy.com
- Fazt Web. Curso de Node.js y Express – API REST completa. Canal de YouTube @FaztWeb.
- midudev. Aprende a crear una API REST con Node.js y Express desde cero. Canal de YouTube @midudev.
- Web Dev Simplified. JWT Authentication Tutorial – Node.js. Canal de YouTube @WebDevSimplified.
- Hola Mundo. Diseño web con CSS Grid y Flexbox. Canal de YouTube @HolaMundoDev.

Artículos y guías técnicas

- CSS-Tricks. A Complete Guide to Flexbox. Disponible en: https://css-tricks.com/snippets/css/a-guide-to-flexbox/
- CSS-Tricks. A Complete Guide to CSS Grid. Disponible en: https://css-tricks.com/snippets/css/complete-guide-grid/
- CSS-Tricks. Custom Properties (Variables) CSS. Disponible en: https://css-tricks.com/a-complete-guide-to-custom-properties/
- JavaScript.info. Fetch API completa. Disponible en: https://javascript.info/fetch
- JavaScript.info. LocalStorage y SessionStorage. Disponible en: https://javascript.info/localstorage
- DigitalOcean. How To Use JSON Web Tokens (JWTs) in Express.js. Disponible en: https://www.digitalocean.com/community/tutorials

Herramientas de desarrollo

- Microsoft. Visual Studio Code. Disponible en: https://code.visualstudio.com
- Ritwick Dey. Live Server Extension para VS Code. Disponible en: https://marketplace.visualstudio.com/items?itemName=ritwickdey.LiveServer
- Oracle Corporation. MySQL Workbench. Disponible en: https://www.mysql.com/products/workbench/
- Postman Inc. Postman API Platform. Disponible en: https://www.postman.com
- Google LLC. Google Fonts – Cinzel, Cinzel Decorative y Raleway. Disponible en: https://fonts.google.com
- Can I Use. Compatibilidad de características web entre navegadores. Disponible en: https://caniuse.com